

NAME:TANKISO MASOEBE

COMPUTER ARCHITECTURE WK 4

1. Scenario Analysis: Gaming Console Hazard

A gaming console can present several hazards if it is not used or maintained properly. One common hazard is overheating. During long gaming sessions, the console produces heat, and if its ventilation areas are blocked by objects, dust, or poor placement, the temperature can rise significantly. Excessive heat may cause the console to shut down unexpectedly, damage internal components, or, in extreme cases, create a fire risk. Another hazard is electrical shock caused by damaged power cables, loose connections, or using an unsuitable power adapter. There is also a physical hazard because cables placed across walkways can cause people to trip and fall. To reduce these risks, the console should be placed in a well-ventilated area, kept away from liquids, and regularly cleaned to prevent dust buildup. Damaged cables should be replaced immediately, and the console should be switched off when not in use. These precautions help ensure safe and reliable gaming.

2. Concept Research: Branch Prediction

Branch prediction is a technique used in modern computer processors to improve performance when executing programs. When a processor encounters a conditional branch, such as an if-else statement, it may not immediately know which instruction should be executed next. Branch prediction allows the processor to predict the likely path before the condition is fully evaluated. If the prediction is correct, the processor continues executing instructions without waiting, reducing delays and improving processing speed. If the prediction is incorrect, the processor discards the incorrectly executed instructions and loads the correct ones. Branch prediction is especially useful in pipelined processors because it helps keep the instruction pipeline full and reduces wasted processing cycles.

3. Tool Practice

```
#include <iostream>

using namespace std;

int main() {
    int temperature;

    cout << "Gaming Console Hazard Simulation" << endl;
```

```

cout << "Enter console temperature in Celsius: ";
cin >> temperature;

if (temperature >= 80) {
    cout << "HAZARD ALERT: Console is overheating!" << endl;
    cout << "Action: Turn off the console and allow it to cool." << endl;
}
else if (temperature >= 60) {
    cout << "WARNING: Console temperature is high." << endl;
    cout << "Action: Check ventilation and remove dust." << endl;
}
else {
    cout << "SAFE: Console temperature is normal." << endl;
}

return 0;
}

```

Reflection

Writing the C++ hazard simulation in Visual Studio Code helped me understand how programming can be used to identify and respond to hazards. I created a program that accepts the temperature of a gaming console and checks whether it is safe, too hot, or overheating. I used if, else if, and else statements to make decisions based on the temperature entered by the user. This practical activity improved my understanding of conditional statements and user input in C++. It also showed me how simple programs can simulate real-world situations and provide warnings. Overall, the exercise improved my programming skills and problem-solving ability.

4. Diagram Design Explanation

A pipeline hazard occurs when an instruction cannot continue normally because it depends on another instruction that has not finished. The diagram shows a five-stage instruction pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The example demonstrates a data hazard

where the second instruction needs a value produced by the first instruction. A stall or bubble is inserted to give the first instruction enough time to produce the required result. Another solution is forwarding, which sends the result directly to the next instruction. Pipeline hazard management helps prevent incorrect results and improves CPU performance by keeping instruction execution organized and efficient.

PIPELINE HAZARD DIAGRAM

A **PIPELINE HAZARD** OCCURS WHEN AN INSTRUCTION DEPENDS ON THE RESULT OF A PREVIOUS INSTRUCTION THAT HAS NOT YET COMPLETED.

INSTRUCTION PIPELINE (5 STAGES)

IF	ID	EX	MEM	WB
Instruction Fetch	Instruction Decode	Execute	Memory Access	Write Back

INSTRUCTION	CLOCK CYCLE								
	1	2	3	4	5	6	7	8	9
I1: ADD R1, R2, R3 (R1 = R2 + R3)	IF	ID	EX	MEM	WB				
I2: SUB R4, R1, R5 (Depends on R1)		IF	ID	STALL (HAZARD)	EX	MEM	WB		
I3: AND R6, R7, R8			IF	ID	EX	MEM	WB		
I4: OR R9, R10, R11				IF	ID	EX	MEM	WB	

DATA HAZARD EXAMPLE

I2 needs the result of I1 (R1), but it is not ready until the end of I1's EX stage.



KEY POINT

Hazards can slow down the pipeline. Techniques like stalling or forwarding help maintain high performance.

HAZARD TYPE



DATA HAZARD

Caused by read-after-write (RAW) dependency. The instruction tries to read a value that is being written by a previous instruction.

SOLUTION 1



STALL / BUBBLE INSERTION

The pipeline inserts a stall (bubble) in I2 during cycle 4, giving time for I1 to complete the EX stage and make the result available.

SOLUTION 2



FORWARDING

Pass the result from I1's EX or MEM stage directly to I2's EX stage without waiting for write-back.